

פרק 4: טכניקות בדיקה - קטגוריות, ניסיון, exploratory ובדיקות קוד

א. קטגוריות בדיקה

• **Specification-based testing (בדיקות מבוססות מפרט)** - טכניקות בדיקה המשתמשות במפרטים (דרישות, תיאור מערכת) כדי לגזור תנאי בדיקה. בדיקות אלו מתמקדות בהתנהגות פונקציונלית (כולל איכותנים כמו ביצועים/אבטחה), ומטרתן למצוא כשלים בהיענות למפרט. רמת K2: K. *זווית מבחן*: כיצד כל טכניקה (למשל בדיקת ערכי קצה, טבלאות החלטה) מאתרת סוג מסוים של שגיאות. *טעות נפוצה*: שימוש רק בטכניקות מבוססות מפרט כאשר יש פגמים בתיעוד או בהבנה - ואז פגמים בקוד שלא לפי המפרט עשויים להישאר לא נתפסים [L42+24-L49].

• **Structure-based testing (בדיקות מבוססות מבנה / קופסה לבנה)** - טכניקות הבודקות את המערכת על סמך מבנה הקוד או הארכיטקטורה שלה (למשל כיסוי שורות קוד, כיסוי מסלולי ביצוע). הן משמשות למדידת כיסוי קוד (כגון *branch coverage* ו-*statement coverage*) ולגילוי לוגיקה פנימית חסרה או שגויה. רמת K2: K. *זווית מבחן*: האם נבדקו כל משפטי הקוד וההחלטות (if/else וכו'), וכיצד מבנה הלוגיקה משפיע על מציאת שגיאות. *טעות נפוצה*: הנחה ש-100% כיסוי משפטי קוד מבטיח כיסוי כל ההחלטות - בפועל במבנה כמו `if (A) print;` `print;` 100% משפטי לא מכסה את שני ענפי התנאי [L79-L87+14].

• **Experience-based testing (בדיקות מבוססות ניסיון)** - טכניקות שמבוססות על ניסיון המבדק והאינטואיציה שלו. דוגמאות: **ניחוש שגיאות**, **בדיקות חוקרות** או **בדיקות מבוססות רשימה**. כאן המבדק משתמש בידע מקדים על המוצר, פגמים קודמים או קטגוריות שגיאות נפוצות (fault taxonomies) כדי לגזור בדיקות. רמת K2: K. *זווית מבחן*: אילו סוגי שגיאות קיימים (לדוגמה, קלטים חריגים, מצב זכירה מלא, הרשאות לא מתאימות) וכיצד הטכניקות הללו ממקדות את הבדיקות כדי לגלותן. *טעות נפוצה*: שימוש בבדיקות מבוססות ניסיון במקום בטכניקות שיטתיות בלבד; לדוגמה, "לחיצות אקראיות" ולא תיעוד שיטתי של השערות שגיאה [L1198-L1207+32-L42-L49+24].

• **Coverage Item (פריט כיסוי)** - יחידת מדידה שמשמשת לחישוב כיסוי הבדיקות. דוגמה: פריט כיסוי יכול להיות *משפט קוד*, *החלטה*, *קבוצה שקילות* או כל מאפיין אחר הניתן למדידה. כיסוי מוגדר כיחס בין מספר הפריטים שנבדקו לעומת כלל הפריטים הקיימים. באופן פורמלי: $Coverage = \frac{\text{מספר פריטים שנבדקו}}{\text{מספר פריטים}} \times 100\%$ [L300-L307+3]. רמת K2: K. *זווית מבחן*: למי שואל לדעת מה בדיוק נמדד בכיסוי ואיך זה משקף רק ממד אחד בלבד של בדיקה (כל פריט מסמל רק היבט אחד) [L300-L307+3]. *טעות נפוצה*: לחשוב שכיסוי 100% מבטיח שהמערכת נבדקה לחלוטין - למעשה זה רק מעיד שכל הפריטים הנמדדים נכנסו לטווח הבדיקה ולא אומר שלא קיימות שגיאות אחרות [L308-L315+3].

• **בחירת קטגוריית בדיקה לפי תיעוד וסיכון** - קבלת החלטה אילו טכניקות לבחור בהתבסס על המידע הקיים והסיכונים בפריקט. למשל, אם יש מפרט מפורט וברור, נעדיף בדיקות מבוססות מפרט (כמו בדיקת קצוות ואקוויולנס); אם הגישה לקוד חשובה או יש סיכון בטיחותי, נוסיף בדיקות מבוססות מבנה; אם התיעוד חלקי או לא קיים, נשען על ניסיון המבדק (במיוחד בדיקות חוקרות וטכניקות ניחוש) כדי לחשוף פגמים "שלא בתוכניות". כמו שצוין במדריך ISTQB: אם חלקים מהמפרט חסרים בקוד, רק בדיקות מבוססות מפרט יגלו זאת, ואם יש קוד שלא במפרט - רק מבוססות מבנה יתפסו את הבעיה [L42-L49+24]. רמת K2: K. *זווית מבחן*: כיצד שילוב טכניקות שונות לפי מקור המידע (מפרט, קוד, ניסיון) מקדם גילוי פגמים משמעותיים. *טעות נפוצה*: הנחה ש"הילחץ הכול בודד תכסה את הכל"; עלולה להוביל לשימוש רק בקטגוריה אחת במקום בשילוב רלוונטי.

ב. ניחוש שגיאות (Error Guessing)

• **Error Guessing (ניחוש שגיאות)** – טכניקת בדיקה מבוססת ניסיון שבה המבדק משתמש בידע שלו (היסטוריית באגים בפרייקט או במערכות דומות) כדי לנסח בדיקות לשגיאות סבירות. הדגש הוא על יצירת היפותזות ספציפיות של פגם ("Fault") ובדיקת תרחישים שעשויים לגרום להם. למשל, ננחש בקלט חריג, שדה ריק או הרשאה חסרה. בחירת השערות מבוססת על קטגוריות שגיאה נפוצות – כגון בעיות קלט (ערכים קיצוניים, ריקים, כפילויות), בעיות הרשאה ואבטחה, מצב מערכת (זיכרון מלא, עומסים), קלט מיוחד (SQL Injection) וכיוצא בזה [L1221+32].

[L1228] . רמת K2: K. *זווית מבחן*: כיצד הטכניקה משלימה טכניקות פורמליות – היא מיועדת לאתר שגיאות שהבדיקות הסטנדרטיות פחות מצפות להן [L1198-L1207+32]. *טעות נפוצה*: לייחס ל-Error Guessing את מלאכת כיסוי הבדיקות ללא תכנון (לטעות ולהניח ש"לחיצה בכל מקום" היא בדיקה מספקת), או לסמן קירוי של פריט כיסוי רק בגלל שפעלנו בצורה רנדומלית. טעות נוספת: ניסיון ניחוש ללא בסיס ניסיון – ניסיונות ניחוש ללא קטלוג פגמים עלולים להיות אקראיים ולא ממוקדים [L1105-L1113+30] [L1125-L1134+30].

• **Fault Attacks (תקיפות פגם)** – גישה מובנית לניחוש שגיאות, שבה המבדק יוצר רשימות של פגמים צפויים (fault hypothesis) ומעצב עבורם בדיקות ממוקדות. למשל, עבור פגם מסוג *קלט קיצון* נריץ בדיקות עם ערכים מחוץ לטווח התקין; עבור *שדה ריק* נבדוק השארת שדה חובה ריק; עבור *כפילויות* נשלח קלטים זהים פעמיים כדי לבדוק טיפול באחסון; עבור *הרשאה* נבדוק כניסה עם משתמש ללא הרשאה מספקת. שימוש ברשימת תקיפות (בתרגום חופשי: רשימת "תקיפות" סבירות) מבסס את הניחושים על דפוסי שגיאה מוכרים, ומשפר את המקצועיות של הטכניקה. רמת K3: K. *זווית מבחן*: בחינה של רכיבים בסיכון גבוה באמצעות "תקיפות" ממוקדות – אילו מצבים בממשק או ברשימת אפשרויות עשויים להוליד כשל. *טעות נפוצה*: להתמקד רק בסוגי קלט רגילים (למשל רק בנתונים טובים) ולהתעלם מסימול מצבים קיצוניים. גם חשיבה בוטה ל"תקיפה" אחת (למצב בלבד) עלולה לפספס עוגנים משמעותיים נוספים.

ג. בדיקות חוקרות (Exploratory Testing)

• **Exploratory Testing (בדיקות חוקרות)** – טכניקת בדיקה שבה הבדיקה מתוכננת, מתבצעת ומוערכת בו-זמנית על-ידי הבודק. המבדק לומד על המערכת תוך כדי, בוחן אותה בצורה חופשית וממקד את המאמץ על תחומים חדשים או לא-מכוסים. בדיקות אלו מתאימות במיוחד במצבים של דרישות חלקיות או סביבה דינמית (Agile), כאשר צריך גמישות וגילוי בעיות שאינן צפויות בתסריטים. לדוגמה, נעסוק בתסריטי שימוש חיים (Use Cases) מורכבים ונחפש באגים של חווית משתמש או לוגיקה שלא נבדקה. רמת K2: K. *זווית מבחן*: הבדל בין בדיקות מבוססות תסריטים (הדקות מראש) לבין בדיקות חוקרות שלא מתעדות מראש כל צעד. בהקשר של מבחן – נבדוק מתי עדיף להשתמש בבדיקה חוקרת (לדוגמה: בשלב תכנון ראשוני של תכונה או בבעיה לא ברורה [L89-L97+41]) ומתי בתסריטים סטנדרטיים (כשקיימים דרישות ברורות והבדיקות השגרתיות קשות לשבירה אוטומטית [L96-L100+44] [L41+L89-L97]). *טעות נפוצה*: להתייחס לבדיקות חוקרות כ"רק לחיצות אקראיות" ללא תיעוד – במקום לתכנן משימה (charter) ולהשתמש במסגרת (Session-Based Test Management) כדי לדווח על כיסוי וממצאים.

• **Charter (צ'רטר בדיקה)** – מגדיר את המטרת הבדיקה לסשן אחד של בדיקה חוקרת. צ'רטר טוב הוא משפט או מספר משפטים קצרים שמתמצים מה לבדוק ובאיזה אופן או למה לשים לב. לדוגמה: "בדוק את תהליך הרישום עם פרטי משתמש שונים כדי לגלות בעיות אבטחה" [L24-L32+54]. צ'רטר חזק ממקד את המבדק ומשלב סיכון מסוים (לרוב ללקוח או לתכונה מרכזית), אך אינו פירוט מלא של צעדים. רמת K2: K. *זווית מבחן*: להבין איך לנסח צ'רטר נכון: מספיק ממוקד (חלוקת המשימה) ומספיק גלובלי (לא להחמיץ תחומים סמוכים). טעות נפוצה: לנסח צ'רטר רחב מדי (כגון "לחקור את כל המערכת" בלי כיוון ברור) – מה שיביא לכיסוי שטחי בלבד [L37-L44+54], או צ'רטר צר מדי שיחסום מסלול בדיקה חשוב.

• **Time Box (הגבלת זמן)** – חלוקת הבדיקה החוקרת לסשנים קצובים בזמן (לרוב 60–120 דקות). בכל ששן קצוב עובד הבודק בתשומת לב מרוכזת על המשימה בצ'רטר. היתרונות: תשומת לב גבוהה (כי הזמן קצוב), יכולת מדידה ברורה (מודדים משך ששן), תכנון קל יותר (יודעים כמה זמן מקדישים), ושמירה על רעננות מנטלית (שילובים של הפסקות). רמת K2: K. *זווית מבחן*: בחינה של פרמטרי הזמן – כמה זמן מתאים לכל session, וכיצד להתמודד עם

הפרעות באמצע הבדיקה (למשל למדוד Session Time מול On-Charter Time ו-Opportunity Time [L163+41]).
[L171] טעות נפוצה: להתעלם משיטת ה-Time Box ולהמשיך לבדוק ללא הפסקה, מה שעלול להוריד איכות הדיווח (השוואת זמנים).

• **Session-Based Test Management (SBTM)** – שיטה המוסדרת לניהול בדיקות חוקרות: מחלקת את הבדיקה לסשנים מזוהים עם צ'רטר, ניהול זמן, ותיעוד ודיון מסודרים. לאחר כל סשן המדווח (Session Report) מתכנסים ב-Debrief משותף (בדו"ח מסקנות) כדי לסכם מה נבדק, אילו באגים נמצאו, וכיצד המשך הבדיקה יתבצע. רמת K: K3. *זווית מבחן:* איך להפוך בדיקה אקראית לניהולית – אילו מרכיבים מדווחים בסשן (כגון: צ'רטר, תוצאות, באגים, זמן) [L209-L218+42] ואילו נושאים נכנסים בדיון הסיכום (סיכום, ממצאים, כיסוי שנותר, צעדים הבאים) [L248-L254+42]. *טעות נפוצה:* לדלג על דיווח סשן – ואז מאבדים עקביות במעקב אחרי באגים וכיסוי. גם להתמקד רק בדיווח על תקלות (pass/fail) במקום ב"תמונה הכוללת" (לדוגמה: אילו תת-מערכות כוסו ומה נשאר) הוא מיני-טעות בתיעוד.

ד. בדיקות מבוססות רשימה (Checklist-Based Testing)

• **Checklist-Based testing (בדיקות מבוססות רשימה)** – טכניקה שבה המבדק יוצר רשימה (צ'ק-ליסט) של תנאי בדיקה או אזורים לבירור, ואז בודק באופן שיטתי כל פריט ברשימה. הפריטים ברשימה נאספים מתוך ניסיון קודם, דרישות חשובות למשתמש, או קטגוריות כשל נפוצות. הרעיון הוא להשוות את רשימת התנאים הנבדקים עם הסיכון במערכת. רמת K: K2. *זווית מבחן:* בניית רשימה מקיפה: למשל כל שדה קלט אפשרי, תנאי קצה, ניהול הרשאות, תהליכים עסקיים עיקריים וכו'. *טעות נפוצה:* הסתמכות על הרשימה כממצה ומוחלטת. רשימה אינה ממצה את כל עולמות הבעיה – אם נשכחים בה מאפיינים (למשל קלטים מיוחדים או תרחישי קצה שלא מופיעים בה), לא נבדקו. גם "סימון סעיף כנבדק" בלי שבוצע בדיקה ממשית (לדוגמה: "ביקשתי הזדהות בתוכנה – תנאי ראשון הסתיים בהצלחה, ולכן סיימתי") היא טעות נפוצה.

ה. בדיקות White-Box בסיסיות (4.3)

• **Statement Coverage (כיסוי משפטי קוד)** – מטרתו לבדוק שכל שורת קוד (משפט קוד) הורצה לפחות פעם אחת במהלך הבדיקות. מדידת כיסוי משפטי נקבעת בתרשים: (מספר משפטים שבוצעו) / סך כל משפטי הקוד) × 100%. לדוגמה, אם בקוד יש 10 שורות, ונכנסנו לכל שורה דרך בדיקה אחת, נקבל 100% כיסוי משפטים. רמת K: K2. *זווית מבחן:* שאלות מתמקדות בהבטחת כל שורה מבוצעת; לפעמים מביאים קטע קוד ונותנים מהו אחוז הכיסוי במצב נתון. *טעות נפוצה:* חשיבה ש-100% משפטים מבטיח כיסוי החלטות (branch). בפועל, במערכת עם if ללא else, אפשר להשיג 100% משפטים גם בלי לבדוק את המקרה הנגדי של ההחלטה [L79-L87+14].

• **Branch/Decision Coverage (כיסוי החלטות / ענפים)** – מטפל בכך שכל נקודת החלטה (למשל תנאי IF) תיבדק לפחות עם ערך True ופעם נוספת עם False. כלומר, כל מסלול לוגי חייב להיבדק בשני הצדדים של תנאי אחד. רמת K: K2. *זווית מבחן:* חישוב כיסוי החלטות לעומת כיסוי משפטים – ידיעת ההבדל; הבנת מקרה קלאסי (למשל שרשרת IF ללא ELSE) שבו 100% כיסוי משפטים אינו מבטיח 100% כיסוי החלטות [L79-L87+14]. *טעות נפוצה:* בבדיקת סנאריו IF/ELSE להתעלם מענף אחד או ללכת רק על תסריט אחד. אם מתכנתים רק מבחן שמגדיר תנאי כפול (למשל A=0) ומדלגים על המקרה שבו A≠0, אז כיסוי ההחלטה לא יהיה מלא.

• **מטרת מדדי כיסוי קוד** – בדיקת סטיקה: מדדי כיסוי כמו statement ו-branch עוזרים לבדוק שדברים לפחות הוצאו לפועל – כלומר, קוד שנכתב לא לבדוק אותו כלל. הם נותנים אינדיקציה לכיסוי אך לא מבטיחים גילוי באגים. ערכם הבסיסי: עמידה בדרישות חוזה בדיקה או ציות לדרישות רגולטוריות (שדורשות למשל כיסוי 100% משפטים בטכנולוגיות קריטיות) [L310-L318+3] [L14+L79-L87]. מגבלה: ייתכן שכל משפט קוד רץ, אך הלוגיקה ביניהם עדיין לא נבדקה. רמת K: K2. *טעות נפוצה:* התייחסות לתוצאות הכיסוי כ"הוכחת איכות". למשל, לחשוב שכיסוי 100% אומר שאין באגים – זה לא נכון.

• **למה זה נבדק גם לבודקת ידנית** – בדיקות קופסה לבנה אינן מיועדות רק למתכנתים: ידע בבדיקות אלו חשוב גם לבודק ידני, כי זה מסייע להבין ולתקשר עם המפתחים (למשל "לאור המדד, צריכים בדיקות נוספות לענף A של הקוד"). בנוסף, בחינות ה-ISTQB כוללות שאלות על מדדי כיסוי (K2) כדי לוודא שהבודק מבין מושגים בסיסיים של

תכנון קוד ובקרה. רמת K1: *זווית מבחן*: שאלות על הגדרה וההבדלים בין סוגי כיסוי קוד, ובהקשר הבוחן: מדוע צריך לבדוק ידני לפחות להבין את המדדים האלה. *טעות נפוצה*: אם טוענים שמי שעוסק רק בבדיקה ידנית לא צריך לדעת כלום על מבנה קוד – בריאלי, להבנה בכל שלבי הפרויקט יש ערך גם במושגים של קוד (למשל תיעוד) בדיקות לאינטגרציה מול מול היחידה בהתאם לכיסוי נדרש).

טעויות נפוצות (Exercises K1-K2)

- פירוש בדיקות חוקרות כ"לחיצות אקראיות" שאינן דורשות שיקול – במקום פירוש סדור, ציור מצע הסשן ותיעוד מה נבדק [L42-L49+24].
- ניסוח **צ'רטר רחב מדי** (למשל "בדוק את המערכת כולה") – מוביל לכיסוי שטחי ול"דילוג" על ממצאים חשובים [L37-L44+54].
- סימון פריט בדיקה ככוסה רק בגלל שניסו משהו בלתי רלבנטי, או משום שרשימת ניחוש ("תיקף את השרת") – ולא מדובר בבדיקה מערכתית אמיתית. למשל, "עשיתי לחיצה בכל שדה" מבלי לבנות בדיקה נגדית.
- בלבול בין **כיסוי משפטים וכיסוי החלטות** – לדוגמה, לחשוב ש-100% כיסוי משפטים מבטיח גילוי כל לולאת IF (המחשה קלאסית של if ללא else) [L79-L87+14].
- שימוש רק ב-*Error Guessing* ללא בדיקות פורמליות – מה שעלול לפספס סוגים אחרים של באגים [L1198+32-L1207].
- שימוש ב-**Checklist** כתחליף למפרט או לסקריפטים – מה שיוצר "סעיפים כלליים מדי" (למשל: "בדוק הרשאה, בדוק תקינות כל השדות" בלי פירוט) והחמצת תרחישים מורכבים.
- חשיבה שבלי רשימת *fault attacks*, הכיסוי בבדיקת ניחוש מספיק – בפועל אין סינון של מה באמת נבדק (משתמשים במחשבה שלילית במקום בגישה מסודרת).
- לקיחת רשימת פריטים לבדיקה ולשכוח לעדכן אותה – למשל, שדה חדש בממשק לא נכלל בה ואם לא שמים לב, לא נבדק כלל.

מבני תרגיל מוצעים

- **כתיבת Chartered Session**: מתן תרחיש (לדוגמה: טופס הרשמה לאפליקציה מובייל חדשה) ובקשה לנסח צ'רטר ממוקד – מה לבדוק (למשל שדה הסיסמה, קלטים מיוחדים) ולמה.
- **חישוב כיסוי החלטות**: קטע קוד פסאודו-קוד עם מבנה IF/WHILE ולא מלא else, ולבקש לחשב את אחוז כיסוי המשפטים וההחלטות לאחר הפעלת מספר בדיקות.
- **בניית Fault Attack**: תיאור שדה קלט (לדוגמה, שדה Username בטופס כניסה) ומילוי רשימת בדיקות של ניחוש שגיאות – למשל: ערך ריק, תווים מיוחדים, התקפות SQL Injection, ניסוי כפילויות (הגשת אותו שם משתמש פעמיים).
- **תסריט הבדלה בין טכניקות**: תיאור מערכת מסוימת (למשל מערכת בנקאית, או ממשק איתותים), ולבקש לסווג אילו טכניקות (spec/struct/exp) יתאימו לבדיקת כל חלק (לדוגמה: ניטור מלא של לוגיקה קריטית בגב כיסוי לבן, בדיקת תזרימי תשלום לפי spec).
- **כרטיס שאלות ייחוס**: הכנת צ'ק-ליסט לבדיקת תרחיש ידני, ואיתור מה חסר בהשוואה לרשימת תנאים מבוססת סיכון.

סגנון שאלות בגרות (דוגמאות – מלכודות מסיחים)

- **בחירה בין קטגוריות**: שאלה רב-ברירתית: "באילו מקרים נעדיף טכניקה מבוססת קוד על פני טכניקה מבוססת מפרט?" – התשובות יכולות לכלול פגיעות אמינות, פגמים בתיעוד או דרישות חוקיות. מלכודת: ציון שמשותף רק כאשר קיים תיעוד מלא (זו תשובה שגויה, מבוססת קוד מתאים גם במצב של תיעוד חלקי).
- **כיסוי קוד**: למשל, "נניח שיש לנו תנאי If ללא Else. איזה מהבאים נכון לגבי כיסוי?" (א) כיסוי 100% משפטים מבטיח כיסוי החלטות, (ב) כיסוי 100% החלטות מבטיח כיסוי משפטים, (ג) 100% משפטים=100% החלטות, (ד) אף לא אחת. הנכונה: (ב) [L79-L87+14] – (הצהרת כיסוי החלטה מקיפה גם את המשפטים שבאינסיד). מושך תשומת לב לסעיף (ג) שהוא לא נכון. [L79-L87+14]
- **צ'רטר versus סקריפט**: למשל, "מה מהבאים אינו מאפיין טיפוס של בדיקה חוקרת?" – (א) אין תסריטים כתובים מראש, (ב) צ'רטר עם מטרה כללית, (ג) תיעוד מדוקדק של כל שלב בזמן, (ד) עיבוד ממצאים לאחר הסשן. תשובה

(ג) מבלבלת - בדיקות חוקרות לעולם לא מתועדות שלב-אחר-שלב על הפרק בזמן הבדיקה אלא רק בקצרים או בדיווח מאוחר.

- **הערכה לפי סיכון:** "בפריקט עם דרישות לא ודאיות, איזו טכניקה עדיפה?" - תשובות דומות, למיון לפונקציה.
- **זיהוי טעויות:** למשל, "איזו מההצהרות הבאות לגבי ניחוש שגיאות נכונה?" (א) מבוסס אך ורק על נסיון המצאות באגים בעבר, (ב) מחליף בדיקות שיטתיות (זה טעות), (ג) מבוסס על השערות נחשיות ונרמזות מראש ע"י רשימות תקיפה (fault attack) [32†L1198-L1207]. התשובה הנכונה: (ג).

הקשר ישראלי

- בראיונות QA בארץ שואלים לעיתים מה ההבדל בין **בדיקה חוקרת** לתסריטית, או מבקשים להגדיר "צ'רטר" כמו בסיילוס הבינלאומי. מונחים נפוצים: "**בדיקות קופסה שחורה/לבנה**" (אך בסלנג המקצועי אומרים גם **-spec-based** או **"white-box"**), "**בדיקת נקודת קצה**" (boundary), "**טעות אנוש**" כשגיאות ברמת המפרט.
- בישראל מביאים דוגמאות מהשטח: למשל, בתחנות Agile נהוג לדבר על "**Sprint Planning**" של בדיקות חוקרות - מי מנסח את הצ'רטרים ואיך. בישיבת **triage** לבאגים הישראלית שואלים לפעמים "מה הסיכון בפגם הזה ומה הצעד הבא בבדיקה?" - וזה מקביל לדיוני ב-Fault Attack ו-SBTM.
- בכתיבה מקצועית בעברית מקובל להשתמש בצירופים מעורבים: למשל "בדיקות חוקרות (Exploratory Testing)", "מערכת קוד פתוח (Open Source)", או מילים כמו "סטל [X] אליה" (סט ל-ife cycle etc - לרוב נהוג להשאיל). ייתכן שמראיינים יבדקו אם המתמודד מכיר את הקיצור SBTM או את ראשי התיבות ISTQB.
- בלוגים ובפורומים ישראליים (כמו TestIL, עולמובדיקות) התייחסו לנושאים אלה; למשל אחד הפודקאסטים של TestIL עסק בהגדרת צ'רטרים לבדיקה חוקרת.

מקורות ורשימות

מקור	מו"ל / כותב	שנה	רישיון
Error - ISTQB Syllabus 4.0.1 (ASTQB) -Guessing, Exploratory, Checklist [5†L217-L225] [5†L242-L249]	ISTQB Foundation (ASTQB)	2018	פרופריטרי (ISTQB) (ASTQB)
אינטרנט: "What are categories of test design techniques?" [18†L55-L63]; "How to choose which technique is best?" [24†L42-L49]	TryQA - לימודי תוכנה	-2018 2020	פרופריטרי (TryQA)
Error Guessing: Experience-Based Testing [32†L1221] [L1198-L1207†32] [L1228]	- Parul Dhingra MasterSoftwareTesting	-2020 2023	פרופריטרי (Parul) (Dhingra)
Session-Based Test Management Guide [42†L248-L254] [L112-L121†41]	Virtuoso QA (Ryan Craven)	2026	פרופריטרי (Virtuoso)
האנציקלופדיה לבדיקות - ITCB / קובי יונסי [L79-L87†14]	ITCB מגזין "עולם הבדיקות"	2016	פרופריטרי (ITCB)
Structure-Based Testing (OODLES blog) [3†L310-L318] [L300-L307†3]	- Oodles Technologies Divya Varnwal	2021	פרופריטרי (Oodles)
Test Charters (מועדון Ministry of Testing) [54†L45-L53] [L24-L32†54]	Ministry of Testing (Dan Ashby)	2024	פרופריטרי (Ministry of Testing)
Coverage Item - ISTQB Glossary [L9]	ISTQB Glossary (עמוד חיפוש)	2023	פרופריטרי (ISTQB)

מקור	מז"ל / כותב	שנה	רישיון
Kaner & Fiedler (Creative Commons) BBST	ספרי לימוד BBST	2014	CC-BY (בליווי קרדיט)
Elisabeth) Test Heuristics Cheat Sheet Hendrickson) 【56†L42-L50】	,Hendrickson, Lyndsay Emery (BBST)	2014	לא ברור
Heuristics (Bach, Satisfice)	James Bach (Satisfice)	2000	לא ברור